

Contents

0.1	子序列问题	2
0.1.1	最长上升/下降/不升/不降子序列 (LIS)	2
0.1.2	最长公共子序列 (LCS)	2
0.2	自然数划分问题	4
0.2.1	可重复	4
0.2.2	不可重复	4
0.3	背包 DP	5
0.3.1	多重背包 (单调队列优化)	5
0.3.2	树上背包	6
0.4	区间 DP	7
0.5	单调队列优化	7
0.6	决策单调性优化	8
0.6.1	二维形式 (2D)	8
0.6.2	一维形式 (1D)	11
0.7	斜率优化	13
0.7.1	斜率优化 (单调队列)	14
0.7.2	斜率优化 (二分队列)	15
0.8	WQS 二分/带权二分优化	17
0.9	动态 DP	18
0.10	状压 DP	19
0.10.1	SOSDP (子集和 DP)	19
0.11	数位 DP	20
0.12	树形 DP	22
0.12.1	换根 DP	22
0.13	关于 DP 后效性的解决思路	24
0.13.1	待定系数法	24
0.13.2	高斯消元法	24
0.13.3	带状矩阵高斯消元	24

0.1 子序列问题

0.1.1 最长上升/下降/不升/不降子序列 (LIS)

我们用 $O(n \log n)$ 的二分做法即可

0.1.2 最长公共子序列 (LCS)

我们有三种考虑

Solution1 首先暴力二维 DP 在 $O(nm)$ 的时间内可以做到求任意两前缀的 LCS

Solution2 如果某一个串的字符不重复，我们可以对两个串进行重构，我们记不会重复的串为 A ，我们按 A 的字母顺序给 B 重新编号，把 B 中未出现在 A 中的字符全删掉，求 B 的 LIS 即可。

1. *Solution3*(源自 *myers* 算法的优化)

我们考虑压位，这样可以优化到 $O(nm/B)$ 的复杂度，具体的说

我们不妨设主串 A ，副串 B ，我们用 $\sum$ 个长度为 m 的 *bitset* 来记录每个字符在 B 串中出现的位置，然后遍历 A 串，我们记前驱 *bitset* 向量（长度为 m ）为 dp ，初始为空，对于当前遍历到的字符 $A[i]$ ，我们考虑它对应的 *bitset*，我们记当前的 $U = dp | pos(A[i])$ ，我们记 $V = (dp << 1) | 1$ ，然后使 $dp' = U \& !(U - V)$ ，此时的 dp' 即为遍历到 i 时的答案向量，然后调用 *popcount* 即可

需要注意的是 *bitset* 减法为二进制向量意义上的减法，而 $!$ 表示取反，由于 *bitset* 不支持减法，我们手动模拟即可。

```
using u64 = unsigned long long;
// 64 位块数表示的 bitset LCS (Hunt-Szymanski 的 bit-parallel 实现)
// 时间  $O(|a| * ((|b|+63)/64))$ ，空间  $O(26 * blocks)$ 
int LCS(const std::string &a, const std::string &b)
{
    int n = (int)a.size(), m = (int)b.size();
    if (n == 0 || m == 0)
        return 0;

    int W = (m + 63) / 64; // 每个 bitset 需要的 u64 个数
    std::vector pos(26, std::vector<u64>(W, 0ULL));

    // 构造 pos: pos[c] 在 b 中字符 c 出现位置的位图
    for (int j = 0; j < m; ++j)
    {
        int c = b[j] - 'a';
        pos[c][j >> 6] |= (1ULL << (j & 63));
    }

    std::vector<u64> dp(W, 0ULL), U(W), V(W), SUB(W);
    // 最后一块只保留到 m 的有效位
    u64 last_mask = (m % 64 == 0) ? ~0ULL : ((1ULL << (m % 64)) - 1ULL);

    for (char ch : a)
    {
        int idx = ch - 'a';

        // U = dp | pos[idx]
        for (int i = 0; i < W; ++i)
            U[i] = dp[i] | pos[idx][i];

        // V = (dp << 1) | 1
        u64 carry = 0;
        for (int i = 0; i < W; ++i)
        {
            u64 next_carry = dp[i] >> 63;
```

```

    V[i] = (dp[i] << 1) | carry;
    carry = next_carry;
}
V[0] |= 1ULL; // 设置最低位

// SUB = U - V (多块减法, 考虑借位)
unsigned long long borrow = 0ULL;
for (int i = 0; i < W; ++i)
{
    // 使用 128 位临时避免加法溢出
    unsigned __int128 v_with_borrow = (unsigned __int128)V[i] + (unsigned __int128)borrow;
    u64 v64 = (u64)v_with_borrow;
    // 结果块
    u64 res = (u64)((unsigned __int128)U[i] - v_with_borrow);
    SUB[i] = res;
    // 产生新的借位当且仅当 v_with_borrow > U[i]
    borrow = (v_with_borrow > (unsigned __int128)U[i]) ? 1ULL : 0ULL;
}

// dp = U & ~SUB
for (int i = 0; i < W; ++i)
    dp[i] = U[i] & ~SUB[i];
// 清除最后块中超出 m 的位
dp[W - 1] &= last_mask;
}

// 统计 dp 中 1 的个数就是 LCS 长度
int ans = 0;
for (int i = 0; i < W; ++i)
    ans += __builtin_popcountll(dp[i]);
return ans;
}

```

0.2 自然数划分问题

我们考虑求 n 个自然数总和为 m 的方案数

我们不妨设 $f(i, j)$ 表示选了 i 个数字总和为 j 的方案数

0.2.1 可重复

Solution1

我们按 n 个数字是否有 1 来讨论

如果有至少一个 1, 那么 $f(i, j)$ 可以由 $f(i-1, j-1)$ 转移而来, 表示我们这一轮选 1

如果没有 1, 那么 $f(i, j)$ 可以从 $f(i, j-i)$ 转移而来, 意思是我们把选的每一个数都手动加上 1

因此转移式为 $f(i, j) = f(i-1, j-1) + f(i, j-i)$

复杂度 $O(nm)$

Solution2

五边形数定理 $O(\text{msqrt}(m))$

```
std::vector<int> calc(int n)
{
    std::vector<long long> a(n), p(n);
    p[0] = 1;
    p[1] = 1;
    p[2] = 2;
    for (int i = 1; i < n; i++) /* 递推式系数 1,2,5,7,12,15,22,26...i*(3*i-1)/2,i*(3*i+1)/2*/
    {
        a[2 * i] = 111 * i * (i * 3 - 1) / 2; /* 五边形数为 1,5,12,22...i*(3*i-1)/2*/
        a[2 * i + 1] = 111 * i * (i * 3 + 1) / 2;
    }
    for (int i = 3; i < n; i++) /*p[n]=p[n-1]+p[n-2]-p[n-5]-p[n-7]+p[12]+p[15]-...+p[n-i*(3i-1)/2]+p[n-i*(3i+1)/2]*/
    {
        for (int j = 2; a[j] <= i; j++)
        {
            p[i] += (j & 2) ? 1 : -1 * p[i - a[j]];
            norm(p[j]);
        }
    }
    return p;
}
```

0.2.2 不可重复

我们仍然用同样的讨论思路

如果要恰有一个 1, 那么我们这一轮选 1 并且让之前的数字都加 1, 也就是 $f(i-1, j-i)$

如果没有 1, 我们直接让之前数字都加 1 即可, 也就是 $f(i, j-i)$

转移式为 $f(i, j) = f(i-1, j-i) + f(i, j-i)$

复杂度 $O(\text{msqrt}(m))$

0.3 背包 DP

0.3.1 多重背包 (单调队列优化)

```
auto multiBag(std::vector<std::array<int, 3>> &goods, int S)
{
    // S 总背包大小
    std::vector<i64> f(S + 1, 0);
    std::vector<int> q(S + 2, 0);
    for (auto [v, w, m] : goods)
    {
        // v 价值, w 体积, m 数量
        auto calc = [&](int i, int j)
        {
            return f[j] + 1ll * (i - j) / w * v;
        };
        for (int up = S; up + w > S; up--)
        {
            int l = 1, r = 0, k = up;
            for (int x = up; x > 0; x -= w)
            {
                for (; k >= std::max(0ll, x - 1ll * m * w); k -= w)
                {
                    while (l <= r and calc(x, k) > calc(x, q[r]))
                        r--;
                    q[++r] = k;
                }
                f[x] = calc(x, q[l]);
                if (q[l] == x)
                    l++;
            }
        }
    }
    return f;
}
```

0.3.2 树上背包

思路往往是考虑每条边的贡献 在上下界优化后复杂度为 $O(n^2)$

我们在每次转移的时候都开一个临时数组 *temp*，目的是防止从同一个子树内重复转移状态

```
template <class T>
class PackageOnTree{
    const T inf = std::numeric_limits<T>::max() / 2;
public:
    std::vector<std::vector<T>> dp;
    void addEdge(int u, int v, T w){
        g[u].emplace_back(v, w);
        g[v].emplace_back(u, w);
    }
    PackageOnTree(int n, int m, int root) : n(n), g(n + 1), siz(n + 1), infCapacity(m), root(root)
    {
        dp.resize(n + 1, std::vector<T>(m + 1));
    }
    void work(){
        dfs(root, 0);
    }
private:
    int infCapacity;
    int n, root;
    std::vector<std::vector<std::pair<int, T>>> g;
    std::vector<int> siz;
    T calc(int u, T w, int cnt) // u 子树内选 cnt 个点 父边权值为 w
    {
        return w * (cnt * (infCapacity - cnt) + (siz[u] - cnt) * (n - infCapacity - (siz[u] - cnt)));
    }
    void dfs(int u, int fa){
        dp[u][0] = 0;
        siz[u] = 1;
        for (auto [v, w] : g[u])
        {
            if (v == fa){
                continue;
            }
            dfs(v, u);
            siz[u] += siz[v];
            std::vector<T> temp(std::min(siz[u], infCapacity) + 1, 0);
            for (int j = std::min(infCapacity, siz[u]); j >= 0; --j)
            {
                for (int k = std::max(0ll, siz[v] + j - siz[u]); k <= std::min(siz[v], j); ++k)
                {
                    temp[j] = std::max(temp[j], dp[u][j - k] + dp[v][k] + calc(v, w, k));
                }
            }
            for (int j = 0; j <= std::min(siz[u], infCapacity); ++j){
                dp[u][j] = temp[j];
            }
        }
    }
};
```

0.4 区间 DP

区间 DP 往往都是 $O(n^3)$ 的复杂度，且操作方式相对单一，我们外层先枚举区间长度，内层枚举左端点，由于长度已知，那么右端点已知，我们枚举中间点转移状态，转移式形如 $dp[i][j] = dp[i][k] + dp[k+1][j] + val$, i, j 为左右端点

0.5 单调队列优化

DP 状态一般为 $dp[i] = \min/\max(dp[j] + cost[j])$

不难发现是个滑动窗口问题，单调队列优化即可

0.6 决策单调性优化

0.6.1 二维形式 (2D)

一般二维的决策单调性 DP 往往是

$$dp[i][k] = \min/\max \sum_{j=1}^{i-1} (dp[j][k-1] + cost[j+1, i])$$

不难发现这种情况的决策单调性来自层与层之间，因为我们的转移也来自不同层

0.6.1.1 分治做法 我们可以用分治的方法解决这个问题，即我们先遍历求出该层 mid 处的最优决策点 然后递归两边即可 复杂度 $nk \log n$, k 为层数

```
template <class T>
class OptimizeDP
{
    const T inf = std::numeric_limits<T>::max() / 2;
public:
    std::vector<std::vector<T>> dp;
    int n, dep;
    OptimizeDP(int n, int k) : n(n), dep(k){
        dp.assign(n + 1, std::vector<T>(k + 1, 0));
        for (int j = 1; j <= k; ++j)
            // 由于转移是层与层之间的 所以我们可以选择分支
            {
                work(1, n, 1, n, j);
            }
    }
private:
    T calcW(int l, int r)
    {
        // 这里我们可以使用正常计算
        // 我们这里可以考虑使用莫队计算 可以证明复杂度是正确的
    }
    void work(int l, int r, int L, int R, int dep) // l 和 r 是求解区间 L,R 是决策区间
    {
        if (l > r)
        {
            return;
        }
        int mid = l + r >> 1;
        T maxVal = 0, transferPoint = -1;
        // 找最大值和转移点
        for (int i = std::min(R, mid); i >= L; --i)
        {
            if (dp[i - 1][dep - 1] + calcW(i + 1, mid) > maxVal)
            {
                maxVal = dp[i - 1][dep - 1] + calcW(i + 1, mid);
                transferPoint = i;
            }
        }
        dp[mid][dep] = maxVal;
        work(l, mid - 1, L, transferPoint, dep); // 根据决策单调性优化
        work(mid + 1, r, transferPoint, R, dep);
    }
};
```


0.6.1.2 二分队列做法 我们同样可以用二分队列的方法解决，具体的说我们开一个三元组队列 队列中 $[j, l, r]$

表示区间在 l, r 的转移最优决策点为 j ，我们只需要动态维护该队列即可

具体的说 我们每次把 $r < i$ 的过时元素弹出，并把队头的 l 改为 i ，更新当前位置 i 的答案。

然后把队尾过时的元素弹出，如果一个元素在 l 处转移仍然没有当前位置 i 更优 它就是过时的。

接下来我们取出队尾，我们二分找出一个 pos 满足 $i < pos$ 时队尾元素更优， $i \geq pos$ 时 i 更优，更新队列即可

复杂度仍为 $nk \log n$ k 为层数

```
template <class T>
class OptimizeDP
{
    const T inf = std::numeric_limits<T>::max() / 2;

public:
    std::vector<std::vector<T>> dp;
    OptimizeDP(int n, int k) : n(n), dep(k)
    {
        dp.assign(n + 1, std::vector<T>(k + 1, inf));
        dp[0][0] = 0;
    }

    void work(int d)
    {
        std::vector<std::array<int, 3>> q(n + 1);
        // {j, l, r} 表示在 [l, r] 内的最优转移点为 j
        int hh = 1, tt = 0;
        q[++tt] = {0, 1, n}; // 初始化
        for (int i = 1; i <= n; ++i)
        {
            while (hh < tt and q[hh][2] < i) // 队头已经过时
            {
                ++hh;
            }
            q[hh][1] = i; // 把队头左端点改为 i
            int k = q[hh][0];
            dp[i][d] = dp[k][d - 1] + getW(k + 1, i); // 转移

            // 如果队尾的决策点转移到 l 从 d 转移的结果不如 i 就不断弹出
            while (hh < tt and getDP(q[tt][0], q[tt][1], d - 1) >= getDP(i, q[tt][1], d - 1))
            {
                --tt;
            }
            q[tt][2] = n; // 队尾的右端点始终为 n
            int l = q[tt][1], r = q[tt][2]; // 二分找到一个位置 后面从 i 转移更优 前面由 tt 转移更优
            int pos = n + 1;
            while (l <= r)
            {
                int mid = l + r >> 1;
                if (getDP(q[tt][0], mid, d - 1) >= getDP(i, mid, d - 1)) // i 更优
                {
                    r = mid - 1;
                    pos = mid;
                }
                else
                {

```

```

        l = mid + 1;
    }
}
if (pos != n + 1)
{
    q[tt][2] = pos - 1;
    q[++tt] = {i, pos, n};
}
}

private:
    int n, dep;
    T getW(int l, int r)
    {
    }
    T getDP(int l, int r, int d) // 层数为 d 从 l 转移到 r 的值
    {
        return dp[l][d] + getW(l + 1, r);
    }
};

```

0.6.2 一维形式 (1D)

一般一维形式的决策单调性 DP 往往是

$$dp[i] = \min/\max \sum_{j=1}^{i-1} (dp[j] + cost[j+1][i])$$

满足决策单调性即 当

$$dp[i] \leq dp[j] \quad j > i \quad J \geq I$$

不难发现这种情况的决策单调性来自同层之间，因为转移来自同层

此时我们将只能选择二分队列做法，因为决策单调性我们是从每层的 mid 开始更新，但是我们发现 1D 形式是同层转移，在我们求 mid 时 $[1, mid - 1]$ 处的 dp 值未知，因此无法分治

0.6.2.1 二分队列做法 具体做法完全和 2D 的二分队列类似，只不过此时我们只需跑一层的转移

复杂度 $n \log n$

```
template <class T>
class OptimizeDP
{
    const T inf = std::numeric_limits<T>::max() / 2;

public:
    std::vector<T> dp;
    OptimizeDP(int n) : n(n)
    {
        dp.assign(n + 1, inf);
        dp[0] = 0;
        work();
    }

    void work()
    {
        std::vector<std::array<int, 3>> q(n + 1);
        // {j, l, r} 表示在 [l, r] 内的最优转移点为 j
        int hh = 1, tt = 0;
        q[++tt] = {0, 1, n}; // 初始化
        for (int i = 1; i <= n; ++i)
        {
            while (hh < tt and q[hh][2] < i) // 队头已经过时
            {
                ++hh;
            }
            q[hh][1] = i; // 把队头左端点改为 i
            int k = q[hh][0];
            dp[i] = dp[k] + getW(k + 1, i); // 转移

            // 如果队尾的决策点转移到 l 从 d 转移的结果不如 i 就不断弹出
            while (hh < tt and getDP(q[tt][0], q[tt][1]) >= getDP(i, q[tt][1]))
            {
                --tt;
            }
            q[tt][2] = n; // 队尾的右端点始终为 n
            int l = q[tt][1], r = q[tt][2]; // 二分找到一个位置 后面从 i 转移更优 前面由 tt 转移更优
            int pos = n + 1;
            while (l <= r)
            {
                int mid = l + r >> 1;
```

```

        if (getDP(q[tt][0], mid) >= getDP(i, mid)) // i 更优
        {
            r = mid - 1;
            pos = mid;
        }
        else
        {
            l = mid + 1;
        }
    }
    if (pos != n + 1)
    {
        q[tt][2] = pos - 1;
        q[++tt] = {i, pos, n};
    }
}

private:
    int n;
    T getW(int l, int r)
    {
    }
    T getDP(int l, int r) // 层数为 d 从 l 转移到 r 的值
    {
        return dp[l] + getW(l + 1, r);
    }
};

```

0.7 斜率优化

斜率优化是一种特殊的优化 DP 其往往表达式为

$$dp[i] = \min/\max(dp[j] + w1[i] + w2[j] + f[i] * g[j])$$

我们稍作变形 $dp[j] + w2[j] = -f[i] * g[j] + dp[i] - w1[i]$

我们把 $dp[j] + w2[j]$ 看做 y , $-f[i]$ 看做 k , $g[j]$ 看做 x , $dp[i] - w1[i]$ 看做 b

那么该式子满足 $y = kx + b$ 的形式

我们想要 $dp[i]$ 的 $\max/\min$ 等价于让 b 的 $\max/\min$

我们不难发现我们转移的每个 j 都对应着一组 (x, y) 也就是说在 k 确定的情况下

我们从很多个 (x, y) 中选择一组使得转移最优

我们推式子看看什么样的 (x, y) 最优 我们以求 $\min$ 为例

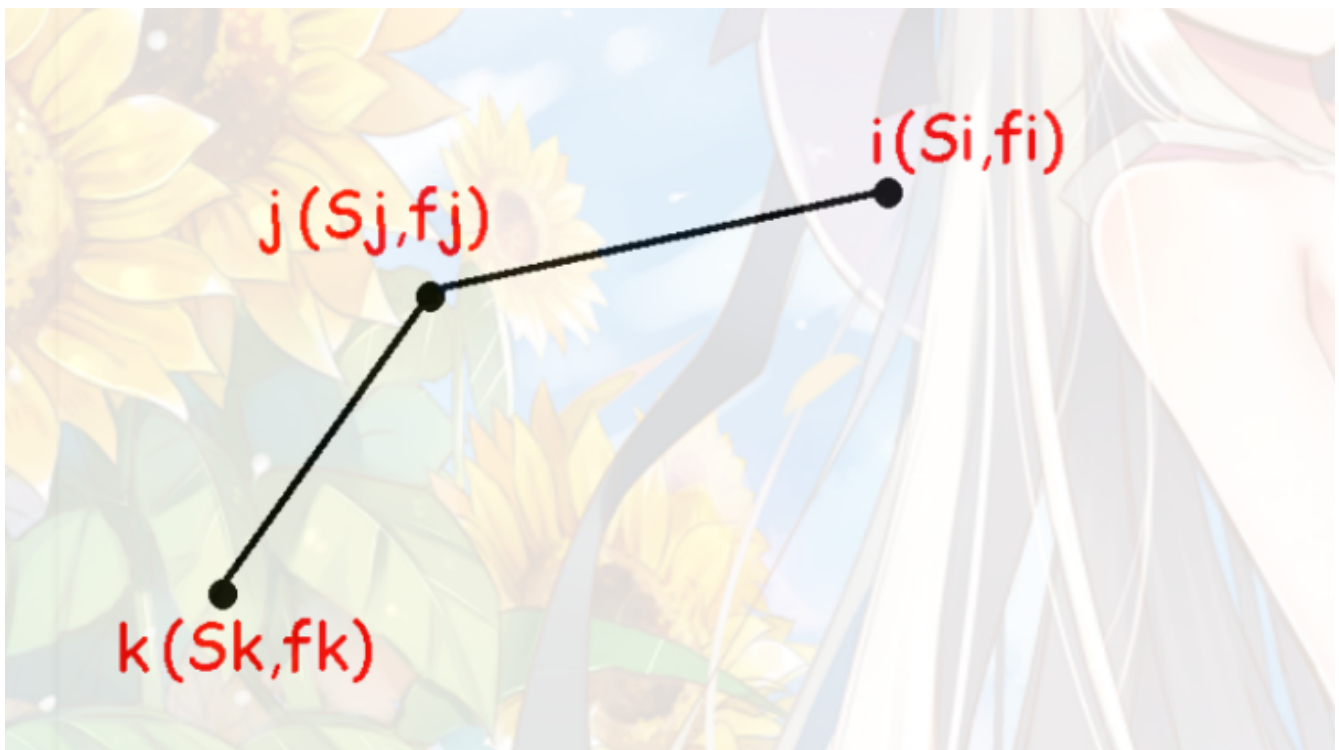
$$b_1 < b_2 \iff y_1 - kx_1 < y_2 - kx_2 \iff \frac{y_2 - y_1}{x_2 - x_1} > k \text{ if } (x_1 < x_2)$$

我们分情况讨论

当 x 存在单调性时

也就是说 $\frac{y_2 - y_1}{x_2 - x_1} > k$ (if $(x_1 < x_2)$) 成立的情况下 $y_2 > y_1$

但是新加点时我们如何决策呢 假如说我们出现了一种情况



我们通过分类讨论可以证明在这种情况下 j 点永远不会成为最优决策点 我们直接弹出即可

有了这个条件, 我们发现维护的队列点集将是一个上凸壳

也就是说队列里的折线斜率单调增加

那么我们可以再分两种情况

如果我们的斜率 k 也单调增加 那么我们发现队头的一些点将会逐渐过时, 我们不断弹出即可, 因此可以单调队列维护 简单来说就是 x 和 k 均单调时可以用单调队列维护

0.7.1 斜率优化 (单调队列)

```
template <class T>
class OptimizeDP{
    const T inf = std::numeric_limits<T>::max() / 2;
public:
    std::vector<T> dp;
    OptimizeDP(int n) : n(n){
        dp.assign(n + 1, inf);
        dp[0] = 0;
        work();
    }
    void work(){
        int hh = 0, tt = -1;
        std::vector<int> q(n + 1, 0);
        q[++tt] = 0;
        for (int i = 1; i <= n; ++i){
            int k = calcK(i);
            while (hh < tt and calcSlope(q[hh], q[hh + 1]) <= k)
            {
                ++hh;
            }
            int j = q[hh]; // 最优转移点
            dp[i] = dp[j] + calcVal(i, j);
            while (hh < tt and calcSlope(q[tt], i) <= calcSlope(q[tt - 1], q[tt]))
            {
                --tt;
            }
            q[++tt] = i;
        }
    }
private:
    int n;
    T calcX(int x)
    {
    }
    T calcY(int x)
    {
    }
    T calcK(int x)
    {
    }
    long double calcSlope(int l, int r){
        if (calcX(r) == calcX(l))
        {
            return inf; // 最大值
        }
        return (long double)((calcY(r) - calcY(l)) / (1.0 * (calcX(r) - calcX(l))));
    }
    T calcVal(int i, int j)
    {
    }
};
```

但是假如说我们的 k 不再单调时,我们将不再能弹出队头,因为我无法预知我们的最优决策点将在哪里出现,于是我们要保留所有的决策点并使其斜率单调,每次在队列里二分找到最优决策点

0.7.2 斜率优化 (二分队列)

```
template <class T>
class OptimizeDP
{
    const T inf = std::numeric_limits<T>::max() / 2;

public:
    std::vector<T> dp;

    OptimizeDP(int n) : n(n)
    {
        dp.assign(n + 1, inf);
        dp[0] = 0;
        work();
    }

    void work()
    {
        int hh = 0, tt = -1;
        std::vector<int> q(n + 1, 0);
        q[++tt] = 0;
        auto findOptimizePoint = [&](int l, int r, int k)
        {
            int ans = r;
            while (l <= r)
            {
                int mid = l + r >> 1;
                if (calcSlope(q[mid], q[mid + 1]) > k)
                {
                    r = mid - 1;
                    ans = mid;
                }
                else
                {
                    l = mid + 1;
                }
            }
            return q[ans];
        };

        for (int i = 1; i <= n; ++i)
        {
            int k = calcK(i);
            int j = findOptimizePoint(1, tt, k); // 在队列里二分找决策点
            dp[i] = dp[j] + calcVal(i, j);
            while (hh < tt and calcSlope(q[tt], i) <= calcSlope(q[tt - 1], q[tt]))
            {
                --tt;
            }
            q[++tt] = i;
        }
    }

private:
    int n;
```

```

T calcX(int x)
{
}
T calcY(int x)
{
}
T calcK(int x)
{
}
long double calcSlope(int l, int r)
{
    if (calcX(r) == calcX(l))
    {
        return inf; // 最大值
    }
    return (long double)((calcY(r) - calcY(l)) / (1.0 * (calcX(r) - calcX(l))));
}
T calcVal(int i, int j)
{
}
};

```

当 x 不存在单调性时，我们的问题将会变的十分麻烦，我们可能需要动态操作凸包，或者利用平衡树，CDQ 分治等高级数据结构来维护凸包。当然我们认为在这种情况下李超线段树可能会成为更好的选择。

0.8 WQS 二分/带权二分优化

在图论中的最小度限制生成树已经有所涉及，WQS 二分能解决的问题往往是在 n 个物品中强制选 m 个的最优方案。我们假设 $f(m)$ 表示选 m 个的最优答案，那么 WQS 二分的使用条件是离散数点函数 $f(m)$ 具有凸性，换句话说 $f(m)$ 的轮廓构成一个凸壳。具有凸性在数学上的定义为二阶导数恒为正或者恒为负，在几何上的体现就是，对于数点 $(m, f(m))$ 构成的图像，相邻两点斜率 $(f[m+1] - f[m])$ 单调。

对于 DP 的优化在于，此类问题往往可能通过 $DP[i][j]$ 表示考虑前 i 个物品选了 j 个的最优结果，在不考虑转移的情况下复杂度为 $O(nm)$ ，那么如果我们使用 WQS 二分复杂度可以降为 $O(n \log \phi)$

在几何上来分析：由于凸壳的斜率单调，我们考虑二分斜率去做凸壳的切线，每条切线都会和凸壳上一个特定的点相切，当切线斜率 k 单调时，切点所对应 x 的坐标也单调，假设我们的切线为 $y = kx + b$ ，那么其中的 y 即为 $f(t)$ ， x 即为 t ，那么式子变为 $f(t) = k * t + b$ ，当然我们目前并不知道 t 是谁。

我们考虑求出 b ，那么 $b = f(t) - t * k$ ，这个式子具有的实际意义是理解的关键。

我们不妨假设我们已经知道 t ，那么式子的意义是在选定 t 个的情况下，把这 t 个元素对应的权值都减掉 k 。那我们怎么通过 k 反向求出 t 呢？我们把所有相关元素的值提前减掉 k ，然后跑朴素的 DP（不限制选多少个）求出此刻选多少个最优，以及最优解为多少，那么此刻选的数量即为 t ，最优解即为 b ，我们所求的 $f(t) = b + t * k$ ，当然这里的 t 可能并不是我们想要的，但我们可以根据单调性去二分调整 k 的值，从而找到我们想要的 t 对应的答案。

0.9 动态 DP

不会，后面补

0.10 状压 DP

0.10.1 SOSDP (子集和 DP)

实际上是解决一类问题，就是说对于所有的 $i \in [0, 2^n - 1]$ ，要求出 $\sum_{j \subset i} a_j$

我们暴力的做法是对每个数都枚举子集，复杂度是 $O(3^n)$

这里给出枚举子集的代码

```
for (int s = 0; s < (1 << n); ++s)
{
    for (int t = s, T = t; T; T = (T - 1) & t) // init 为要枚举的对象
    {
        // 对应的子集为 T
        dp[s] += dp[T];
    }
}
```

然后我们发现每次枚举都经过了很多次同样的位置，我们想到对这些位置统一处理，也就是只经过一次

那么我们可以写出 SOSDP 优化后的代码

```
for (int j = 0; j < n; ++j)
{
    for (int i = 0; i < (1 << n); ++i)
    {
        if (i >> j & 1)
        {
            dp[i] += dp[i ^ (1 << j)];
        }
    }
}
```

当然我们不局限于求子集，也可以做到求超集，也可以从原数向子集 DP 等等

0.11 数位 DP

我们发现数位 DP 的一般形式为按位考虑，然后综合考虑所有限制情况来 DP

我们用记忆化搜索的方法搞定数位 DP，我们给出一般形参的可能情况

以下为记忆化搜索函数 *dfs* 的常设定的形参

- *pos*: int 型变量，表示当前枚举的位置，一般从高到低
- *limit*: bool 型变量，表示枚举的第 *pos* 位是否受到限制，
 - 为 true 表示取的数不能大于 $a[pos]$ ，而只有在 $[pos + 1, len]$ 的位置上填写的数都等于 $a[pos]$ 时该值才为 true
 - 否则表示当前位没有限制，可以取到 $[0, k-1]$ ，因为 k 进制的数中数位最多能取到的就是 $k - 1$
- *last*: int 型变量，表示上一位（第 $pos + 1$ 位）填写的值
 - 往往用于约束了相邻数位之间的关系的题目
- *lead0*: bool 型变量，表示是否有前导零，即在 $len - > pos + 1$ 这些位置是不是都是前导零
 - 基于常识，我们往往默认一个数没有前导零，也就是最高位不能为 0，即不会写为 000123，而是写为 123
 - 只有没有前导零的时候，才能计算 0 的贡献。
 - 那么前导零何时跟答案有关？
 - * 统计 0 的出现次数
 - * 相邻数字的差值
 - * 以最高位为起点确定的奇偶位
- *sum*: int 型变量，表示当前 $len - > pos + 1$ 的数位和
- *r*: int 型变量，表示整个数前缀取模某个数 m 的余数
 - 该参数一般会用在：约束中出现了能被 m 整除
 - 当然也可以拓展为数位和取模的结果
- *st*: int 型变量，用于状态压缩
 - 对一个集合的数在数位上的出现次数的奇偶性有要求时，其二进制形式就可以表示每个数出现的奇偶性

```
class DigitDP
{
public:
    DigitDP(int infLength, int base) : a(infLength + 1), base(base)
    {
        f.assign(); // 视题目而定
    }

    int solve(int l, int r)
    {
        return calc(r) - calc(l - 1);
    }

private:
    std::vector<int> a;
    std::vector f; // 开几个维度视情况而定
    int base; // 进制基数
    // pos 是第几位 我们从高到低枚举 0 为递归出口 limit 表示这一位是否受限制 lead0 表示前面是否都为前导 0
    // r 表示 len 位之前的数 modp 的余数 last 表示上一个数字
    void dfs(int pos, int limit, int lead0, int r, int last, int sum)
    {
        if (pos == 0)
        {
            if (满足条件)
            {
                return 1 or sum; // 1 是数量 sum 是该数字的属性
            }
        }
        if (!limit and f[pos][ ][ ] != -1)
        {
```

```

        // 不受限制且后面的结果已经求得
        return f[pos][][];
    }
    int up = limit ? a[pos] : base - 1; // 数字的上限
    int res = 0;
    for (int i = 0; i <= up; ++i)
    {
        if (lead0 and i == 0) // 考虑全都是 0 会怎么样
        {
            res += dfs(pos - 1, limit and i == up, lead0 and i == 0, .....);
        }
        else
        {
            if (满足条件)
            {
                res += dfs(pos - 1, limit and i == up, lead0 and i == 0, .....);
            }
        }
    }
    if (!limit) // 不受限制
    {
        f[pos][][] = res;
    }
    return res;
}

int calc(int x)
{
    int len = 0;
    while (x)
    {
        a[++len] = x % base; // k 是进制数 一般为 10 或者 2
        x /= base;
    }
    return dfs(len, 1, .....); // 数位 dp 记忆化搜索过程 形参因题而定
}
}

```

0.12 树形 DP

0.12.1 换根 DP

首先换根 DP 分两种情况，边换根和点换根。

边换根 DP 并不常见，且处理手法类似，我们主要考虑点换根 DP。

然后我们又有两种思考方式来做换根 DP

一种是两次 DFS 用一个数组，最终的 DP 数组即为答案

另一种是两次 DFS 分别用一个数组，其中 $F(u)$ 表示以 u 为根子树内的答案, $G(u)$ 表示以 u 为根子树外的答案，我们将两个数组拼接之后才是真正的答案

0.12.1.1 前后缀积优化 其实是一个常见的换根 DP Trick，也就是说当我们的 DP 转移如果是关于子树的乘积并涉及取模时，我们换根难免会涉及到除法以及乘法逆元，但当 mod 数不是质数时逆元不存在，或者说求逆元的值取模后为 0。我们将无法处理这个问题，于是我们考虑不再做除法，而是利用前后缀积来优化这个问题。

我们以转移式为 $f[u] = \prod_{v \in \text{son}[u]} (f[v] + 1)$ 为例子

```
template <class T>
class PreSufOptimizeDP{
public:
    std::vector<T> dp;
    PreSufOptimizeDP(std::vector<std::vector<int>> &g) : g(g), dp(g.size()), pre(g.size()), suf(g.size())
    {
        dfs1(1, 0);
        dfs2(1, 0);
    }
private:
    std::vector<std::vector<int>> g;
    std::vector<std::vector<T>> pre, suf;
    void dfs1(int u, int fa){
        dp[u] = 1;
        pre[u].push_back(1);
        for (auto v : g[u]){
            if (v == fa)
            {
                continue;
            }
            dfs1(v, u);
            dp[u] = dp[u] * (dp[v] + 1);
            pre[u].push_back(dp[v] + 1);
            suf[u].push_back(dp[v] + 1);
        }
        suf[u].push_back(1);
        for (int i = 1; i < pre[u].size(); ++i)
        {
            pre[u][i] = pre[u][i] * pre[u][i - 1];
        }
        for (int i = suf[u].size() - 2; i >= 0; --i)
        {
            suf[u][i] = suf[u][i] * suf[u][i + 1];
        }
    }
    // dp[v]=dp[v]*(dp[u]/(dp[v]+1)+1);
    void dfs2(int u, int fa)
    {
        int idx = 0;
        for (auto v : g[u]){
            if (v == fa)
            {
                continue;
            }
            dp[v] = dp[v] * (pre[u][idx] * suf[u][idx + 1] + 1);
            ++idx;
            dfs2(v, u);
        }
    }
};
```

0.13 关于 DP 后效性的解决思路

我们的 DP 方程有时候并不能直接递推，他们的转移可能存在环路，这类问题的解决方法一般是把 DP 式子当做方程，通过待定系数法或高斯消元法解决

0.13.1 待定系数法

待定系数法最经典的应用是树上随机游走问题，因为每次随机游走既能走到儿子，又能走到父节点，因此 DP 显然有环。

以一个题目为例，我们从根节点开始，每次随机走向一个邻点，问走到叶子的期望游走次数，那么我们很容易写出转移方程

$$DP(x) = \frac{1 + \sum DP(N(x))}{|N(x)|}, \text{ 其中 } N(x) \text{ 是树上 } x \text{ 点的邻域, 如果我们把父节点分开, 即 } DP(x) = \frac{1 + DP(fa[x]) + \sum DP(son(x))}{|N(x)|}$$

我们把 $DP(son(x))$ 代入原 DP 式子，解得

$$DP(x) = \frac{1}{|N(x)| - \sum k_y} * DP(fa[x]) + \frac{1 + \sum b_y}{|N(x)| + \sum k_y}$$

，我们发现解出的 k_x b_x 只和 k_y 和 b_y 有关，因此我们可以递推求出所有节点的 k 和 b ，对于根节点，由于其没有父节点 $DP(root) = b_{root}$ ，求解完成

对于待定系数法的使用，我们并没有特定的标准，只能通过选手自己观察式子特点灵活操作

0.13.2 高斯消元法

待定系数法往往只能解决特殊性质的题目，高斯消元则更加朴素，能在 $O(n^3)$ 时间内解决线性递推的方程组

仍然以一个题目为例： n 个点 m 条边的无向简单连通图，从 1 号点开始随机游走，到 n 号点时停止，问每个点的期望经过次数。在随机游走中，由于树有极好的性质，我们可以利用待定系数，但图并不具有树的性质，在数据范围允许的情况下，就要考虑高斯消元法

我们写出状态转移方程

$$DP(1) = \sum_{(1,j) \subseteq E} \frac{DP(j)}{\deg(j)} + 1$$

$$DP(i) = \sum_{(i,j) \subseteq E} \frac{DP(j)}{\deg(j)}$$

$$DP(n) = 1$$

起点和终点由于其特殊性转移不同，我们发现可以对 $[1, n - 1]$ 的式子进行高斯消元法解出 $n - 1$ 个值，从而得到结果

0.13.3 带状矩阵高斯消元

是高斯消元法的特殊情况，也就是说，增广矩阵可能具有特殊性质，比如只有一部分区域可能存在非 0 系数，我们把这类矩阵称作带状矩阵，如果带宽是 d 的话，那么复杂度为